ADAN8888 — Week 8: Develop Third Modeling Approach

===== WEEK 8 START =====

Goal: Forecast next-month passenger demand for U.S. domestic carrier–route–month observations from official BTS T-100 data, using a third modeling family: **feedforward multilayer perceptron (MLP) regression**.

Continuity: The exact Week 3 chronological split and Week 5 training-fitted PCA features (`PC1` , `PC2`) are preserved. Week 6 Ridge and Week 7 HGB_C provide historical validation benchmarks. No test targets are scored or used for selection.

Assignment design: Train **three** neural architectures; report training and validation MAE, RMSE and R^2 (passenger counts); choose the Week 8 winner by validation MAE. Where exact ties occur, use validation RMSE and fewer trainable weights. A lower validation score is better.

Code architecture: All modeling functions found in `Src/week8_src_mlp_neural_network.py` .

Important:

- Week 8 is a separate third model family, and more architecture complexity does **not** guarantee outperforming Week 7.
- Al true metrics must be generated locally from the original data.

1. Load Week 8 code and identify project paths

In [15]:
```
from pathlib import Path
import importlib.util
import sys
import numpy as np
import pandas as pd
from IPython.display import display, Image

def find_root():
    here = Path.cwd().resolve()
    for location in (here,*here.parents):
        if (location / "Src" / "week8_src_mlp_neural_network.py").is_file():
            return location
    raise FileNotFoundError("Open the existing repository root or Notebooks folder.")

ROOT = find_root()
SOURCE = ROOT / "Src" / "week8_src_mlp_neural_network.py"
spec = importlib.util.spec_from_file_location("week8_mlp",SOURCE)
week8 = importlib.util.module_from_spec(spec)
sys.modules[spec.name] = week8
spec.loader.exec_module(week8)
REPORTS = ROOT / "Reports" / "week8"
TABLES = REPORTS / "week8_tables"
FIGURES = REPORTS / "week8_figures"
print("Project root:", ROOT)
print("Week 8 reusable code:", SOURCE)
print("Candidate models:", len(week8.CANDIDATES))
```

```
Project root:
Week 8 reusable code:

Candidate models: 3
```

2. Reconstruct full PC1/PC2 training and validation data

In [16]:
```
data, split_summary, source_description = week8.load_full_training_validation(ROOT)
display(split_summary)
print("Input provenance:", source_description)
assert set(data) == {"Training", "Validation"}
assert len(data["Training"]) == 1_062_603
assert len(data["Validation"]) == 239_395
assert pd.to_datetime(data["Training"].ForecastMonth).max() < pd.to_datetime(data["Validation"].ForecastMonth).min()
print("Verified exact temporal separation and full-size datasets.")
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	Mo
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	

```
Input provenance: Week 5 source (reconstructed full PCA from official BTS ZIPs)
Verified exact temporal separation and full-size datasets.
```

3. Third modeling approach: neural networks and scaling

A feedforward neural network learns nonlinear transformations with trained weights in multiple hidden layers. `MLPRegressor` uses Adamm optimization and nonlinear ReLU activations; compared with Week 7 boosted trees, it is a distinct modeling family with multiple layers and non-convex training.

Model	Hidden-layer architecture	Controlled comparison
MLP_A	(32,16)	Smaller two-hidden-layer starting point
MLP_B	(64,32)	Wider layers than A
MLP_C	(64,32,16)	Adds a third hidden layer beyond B

- Shared settings: `alpha=0.001`, `solver='adam'`, `learning_rate_init=0.001`, `batch_size=4096`, `max_iter=45`, `random_state=42`, `early_stopping=False`.
- Solver can stop using **training loss alone**.
- No random validation holdout is used.

- PCA features standardized on training only in a `Pipeline`.
- Training target values standardized in a `TransformedTargetRegressor` and predictions are returned to the original passenger scale before MAE, RMSE, and R^2.
- All negative predictions clipped to zero (matchs prior-week reporting).

In [17]:
```
experiment = pd.DataFrame([
    {"Model":c["Model"], "HiddenLayers":"-".join(map(str,c["hidden_layer_sizes"])),
     "TrainableParameters":week8.parameter_count(c["hidden_layer_sizes"])}
    for c in week8.CANDIDATES
])
display(experiment)
print("Fixed hyperparameters:",week8.COMMON)
assert len(experiment)==3
assert list(experiment.TrainableParameters)==sorted(experiment.TrainableParameters.tolist())
```

	Model	HiddenLayers	TrainableParameters
0	MLP_A	32-16	641
1	MLP_B	64-32	2305
2	MLP_C	64-32-16	2817

```
Fixed hyperparameters: {'activation': 'relu', 'solver': 'adam', 'alpha': 0.001, 'batch_size': 4096, 'learning_rate_init': 0.00
1, 'max_iter': 45, 'tol': 0.0001, 'n_iter_no_change': 10, 'early_stopping': False, 'random_state': 42, 'shuffle': True}
```

4. Fit all three candidate neural networks

Each candidate fits on the **full training partition** and is evaluated separately using the full training and subsequent validation partitions.

In [18]:
```
result = week8.fit_three_variations(
    data["Training"], data["Validation"], split_summary, source_description
)
display(result.comparison.round(4))
assert len(result.comparison)==3
assert len(result.metrics_long)==6
```

```
Finished MLP_A (32-16); validation MAE=1,171.51; epochs=20
Finished MLP_B (64-32); validation MAE=1,143.20; epochs=17
Finished MLP_C (64-32-16); validation MAE=1,149.90; epochs=17
```

	Model	HiddenLayers	TrainableParameters	FittedEpochs	TrainingConverged	Training_MAE	Training_RMSE	Training_R2	Validation_MAE
0	MLP_A	32-16	641	20	True	1171.3749	2236.7079	0.9169	1171.5124
1	MLP_B	64-32	2305	17	True	1161.6278	2236.2033	0.9169	1143.1952
2	MLP_C	64-32-16	2817	17	True	1159.0139	2230.2336	0.9174	1149.8986

5. Analyze generalization, optimize against validation MAE and select the winner

- A more complex architecture could reduce training loss potentially and worsening validation error.
- However, A simple validation-minus-training gap cannot by itself establish overfitting because time periods can differ in traffic mix.
- We therefore compare all **three** MAE, RMSE, and R^2 outcomes and acknowledge trade-offs across metrics.

In [19]:
```
display(result.ranking[["Model","HiddenLayers","TrainableParameters","FittedEpochs",
                        "Training_MAE","Validation_MAE","Training_RMSE","Validation_RMSE",
                        "Training_R2","Validation_R2","MAE_Gap_ValidationMinusTraining"]].round(4))
print("Validation-MAE-selected Week 8 winner:",result.selected_name)
print("Chosen architecture:",result.settings["hidden_layer_sizes"])
print("Week 7 HGB_C validation MAE benchmark:",f"{week8.WEEK7_VALIDATION_MAE_REFERENCE:,.2f}","passengers")
assert result.selected_name==result.ranking.iloc[0].Model
```

	Model	HiddenLayers	TrainableParameters	FittedEpochs	Training_MAE	Validation_MAE	Training_RMSE	Validation_RMSE	Training_R2	V
0	MLP_B	64-32	2305	17	1161.6278	1143.1952	2236.2033	1988.7780	0.9169	
1	MLP_C	64-32-16	2817	17	1159.0139	1149.8986	2230.2336	1999.7847	0.9174	
2	MLP_A	32-16	641	20	1171.3749	1171.5124	2236.7079	2019.4310	0.9169	

```
Validation-MAE-selected Week 8 winner: MLP_B
Chosen architecture: (64, 32)
Week 7 HGB_C validation MAE benchmark: 1,161.07 passengers
```

6. Save reproducible results

In [20]:
```
week8.save_results(result,ROOT)
print((REPORTS/"week8_verified_results_for_report.md").read_text(encoding="utf8"))
print("Models:",[x.name for x in (ROOT/"Models"/"week8").glob("*")])
```

```
# Week 8 — Empirical Results (See written report)

| Model | Hidden layers | Parameters | Epochs | Train MAE | Val MAE | Train RMSE | Val RMSE | Train R² | Val R² |
|---|---|---|---|---|---|---|---|---|---|
| MLP_A | 32-16 | 641 | 20 | 1,171.37 | 1,171.51 | 2,236.71 | 2,019.43 | 0.9169 | 0.9380 |
| MLP_B | 64-32 | 2,305 | 17 | 1,161.63 | 1,143.20 | 2,236.20 | 1,988.78 | 0.9169 | 0.9399 |
| MLP_C | 64-32-16 | 2,817 | 17 | 1,159.01 | 1,149.90 | 2,230.23 | 1,999.78 | 0.9174 | 0.9392 |

**Week 8 selected: MLP_B**, validation MAE **1,143.20 passengers**.
Week 7 HGB_C reference validation MAE: 1,161.07 passengers.
**Test partition evaluated: NO.**


See `week8_figures/` for generated figures.

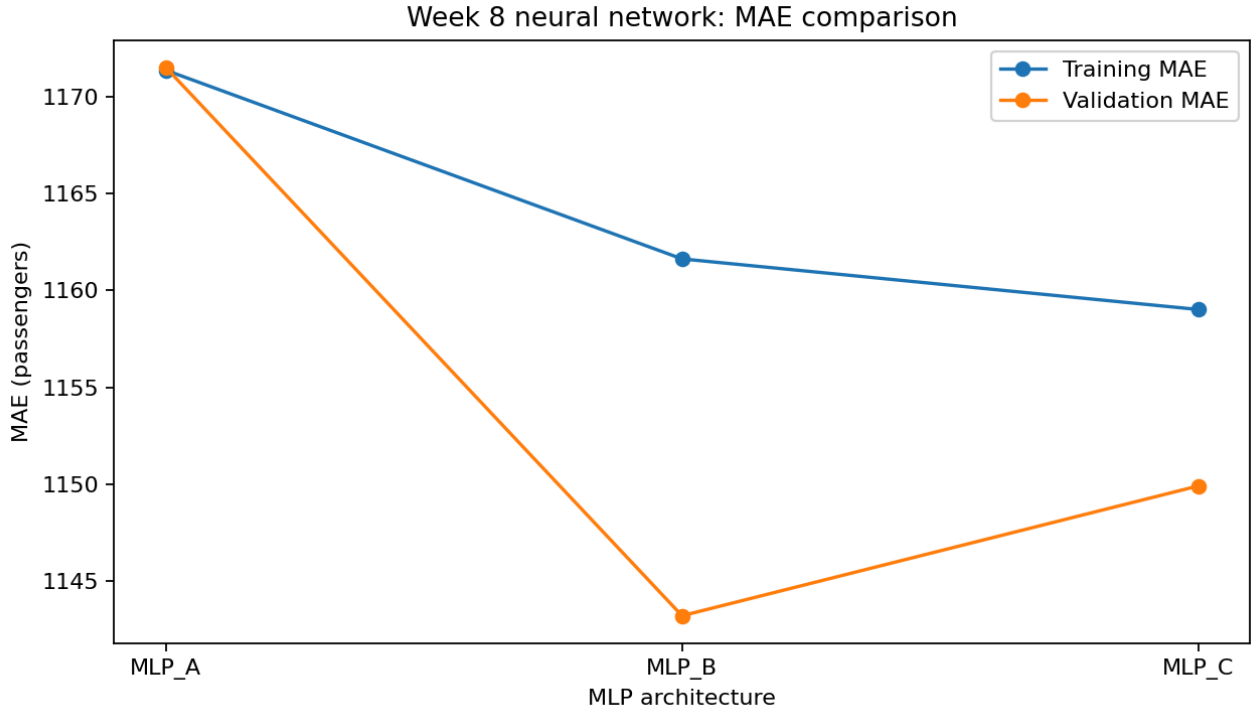
Models: ['README.md', 'week8_model_manifest.json', 'week8_selected_mlp_neural_network.joblib']
```

7. Review metric, prediction, neural-optimization figures

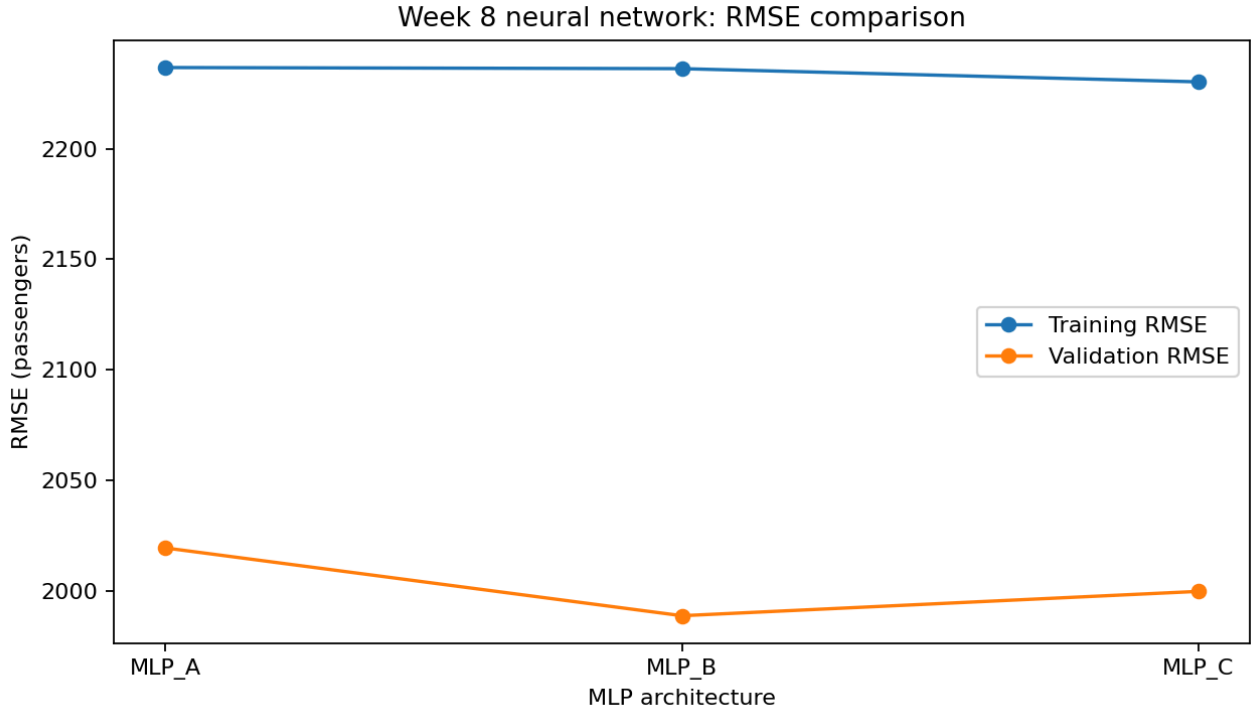
- MAE/RMSE comparisons and monthly validation MAE use the whole dataset.
- Actual-versus-predicted scatter displays 4,000 reproducibly sampled rows only; all numerical metrics use full partition s.
- Optimization loss is on the **scaled target** (not passenger-count MAE).

In [21]:
```
for filename in [
    "week8_figure_mae_comparison.png", "week8_figure_rmse_comparison.png",
    "week8_figure_validation_actual_vs_predicted.png",
    "week8_figure_monthly_validation_mae.png", "week8_figure_training_loss_curves.png"
]:
    image_file = FIGURES / filename
    assert image_file.is_file(),f"Missing figure {image_file}"
    print(filename)
    display(Image(filename=str(image_file)))
```

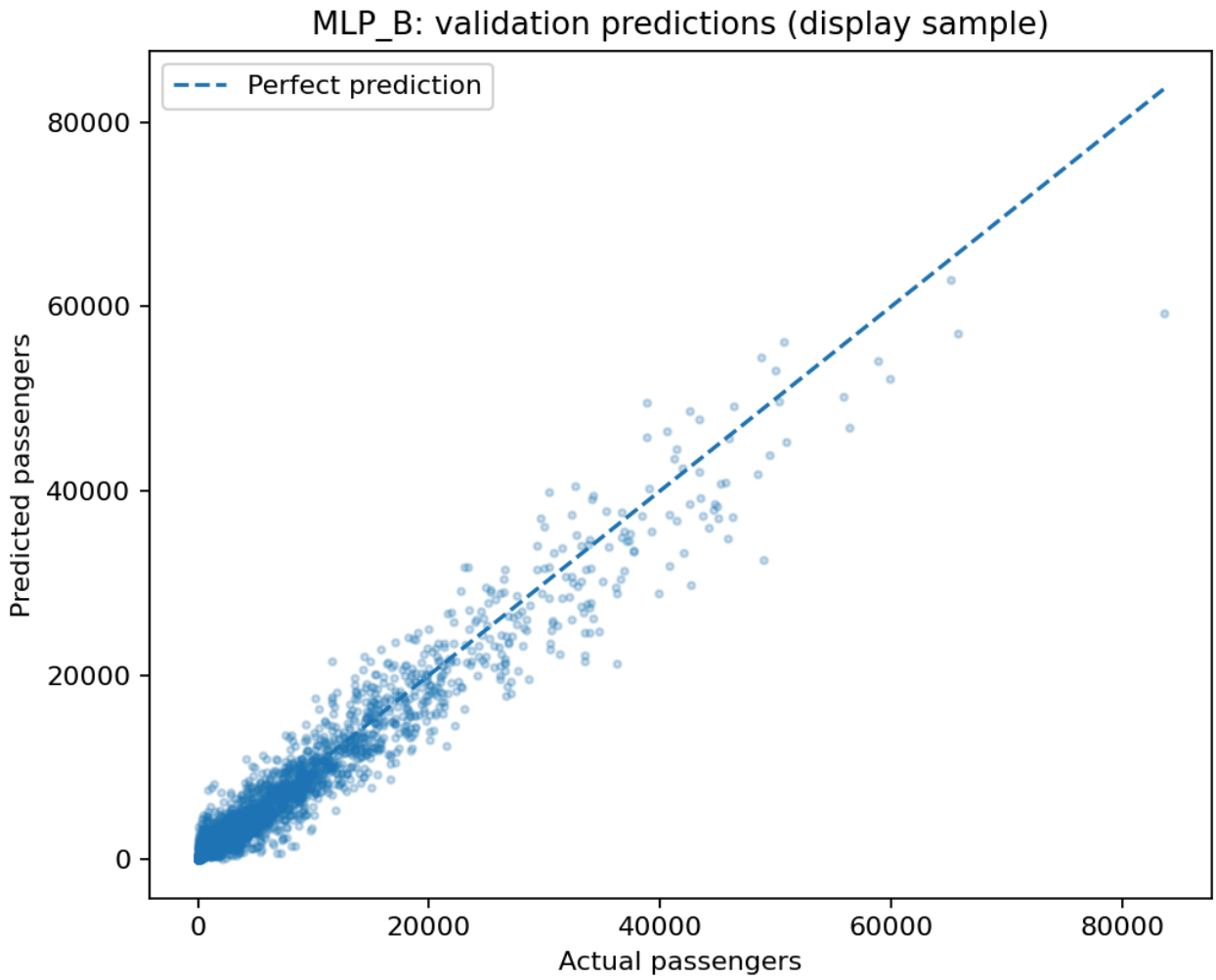
week8_figure_mae_comparison.png



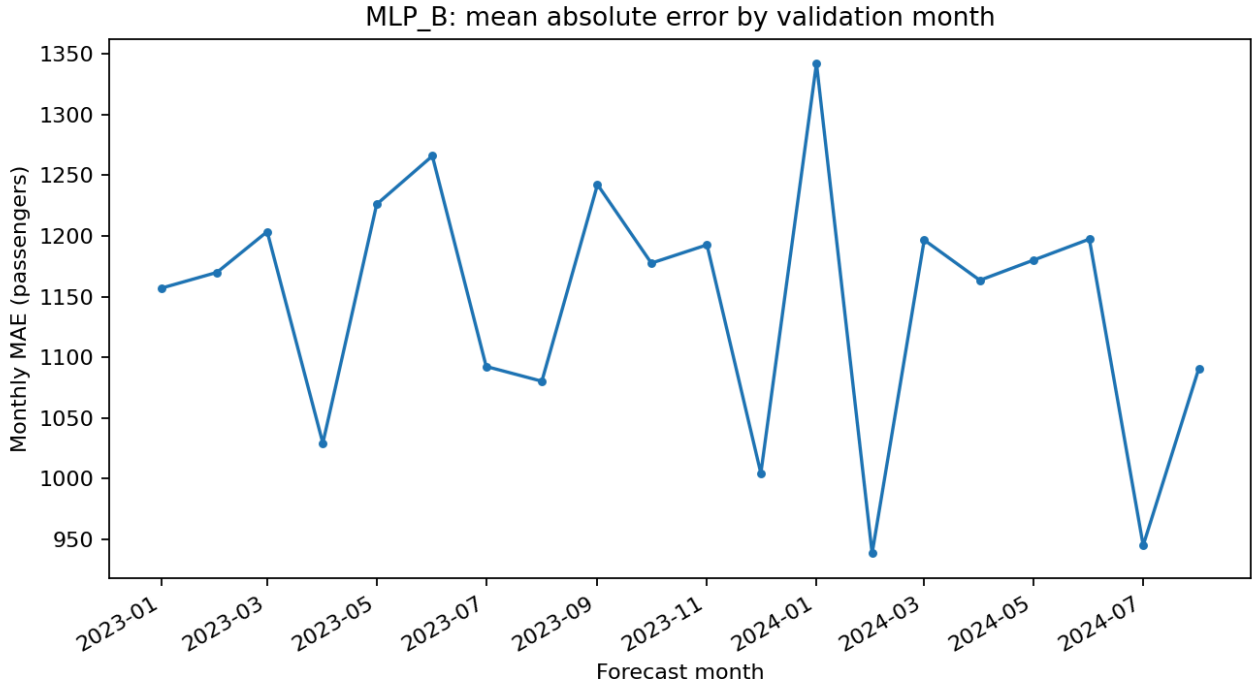
week8_figure_rmse_comparison.png



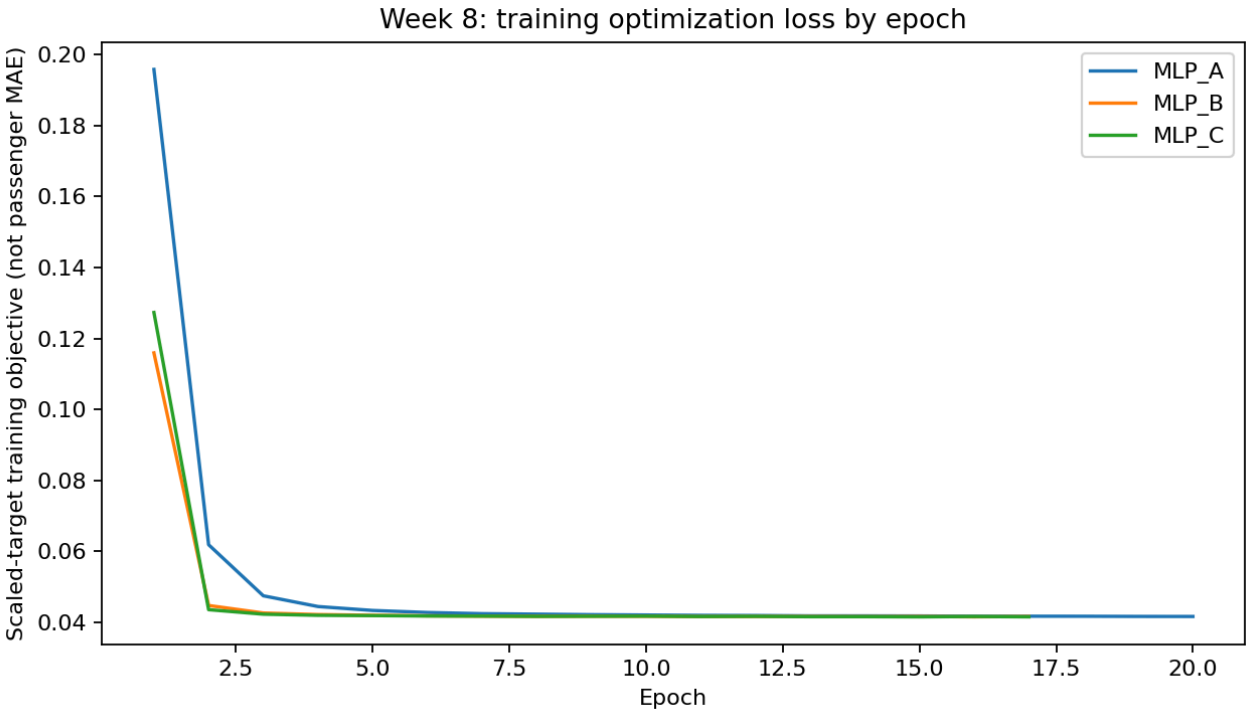
week8_figure_validation_actual_vs_predicted.png



week8_figure_monthly_validation_mae.png



week8_figure_training_loss_curves.png



8. Final audit and end marker

In [22]:
```
assert len(pd.read_csv(TABLES/"week8_three_model_comparison.csv"))==3
assert len(pd.read_csv(TABLES/"week8_metrics_long.csv"))==6
assert (TABLES/"week8_winner_summary.json").is_file()
assert (ROOT/"Models"/"week8"/"week8_selected_mlp_neural_network.joblib").is_file()
for col in ("Training_MAE","Validation_MAE","Training_RMSE","Validation_RMSE","Training_R2","Validation_R2"):
    assert np.isfinite(result.comparison[col]).all()
assert result.selected_name in {"MLP_A","MLP_B","MLP_C"}
print("All Week 8 checks passed.")
print("===== WEEK 8 END =====")
```

```
All Week 8 checks passed.
===== WEEK 8 END =====
```